

Politechnika Rzeszowska
Wydział Matematyki i Fizyki Stosowanej

Sprawozdanie

Alorytmy i struktury danych

ZADANIE PROJEKTOWE/ALGORYTMICZNE

Autor:
MATEUSZ DZIAŁOWSKI

20 stycznia 2026

Spis treści

1	Wstęp.	2
2	Opis problemu.	3
3	Podstawy teoretyczne.	3
3.1	Złożoność algorytmu teoretyczna.	3
4	Szczegóły implementacji.	3
4.1	Rozwiązanie problemu - podejście pierwsze.	3
4.1.1	Analiza problemu.	3
4.1.2	Schemat blokowy algorytmu	5
4.1.3	Algorytm zapisywany w pseudokodzie.	6
4.1.4	Sprawozdanie poprawności algorytmu.	7
4.1.5	Teoretyczne oszacowanie złożoności obliczeniowej.	7
4.2	Rozwiązanie problemu - podejście drugie.	7
4.2.1	Schemat blokowy algorytmu.	8
4.2.2	Algorytm zapisywany w pseudokodzie.	9
4.2.3	Sprawdzenie poprawności algorytmu.	9
4.2.4	Teoretyczne oszacowanie złożoności obliczeniowej.	9
4.3	Implementacja algorytmów w języku C++ oraz Cmake wraz z Clang++.	10
4.4	Implementacja	10
4.4.1	Kod źródłowy	10
4.4.2	Testy niewygodnych zestawów danych.	11
4.4.3	Test wydajnościowy algorytmów.	13

1 Wstęp.

Celem niniejszego opracowania projektu jest przedstawienie implementacji oraz analizy wydajnościowej algorytmu, rozwiązującego problem wyszukiwania specyficznych podciągów w zadanym ciągu liczb całkowitych dodatnich wejściowych. Dokumentacja obejmuje szczegółowy opis problemu, podstawy teoretyczne, implementację algorytmu oraz wyniki testów wydajnościowych przeprowadzonych na różnych zestawach danych wejściowych.

"W przedstawionym opracowaniu indexowanie pozycji w podciągach i tablicach zaczyna się od 0."

2 Opis problemu.

Dla zadanego ciągu liczby całkowitych dodatnich ciągu $A[]$ o długości N , należy znaleźć wszystkie podciągi o długości $M = 5$ (pięcioelementowe) w danym ciągu, które spełniają kryterium: Suma elementów na pozycji 2 i 4 jest większa od sumy elementów na pozycji 1, 3 i 5.

Dla, podciągu $T[] = [t_0, t_1, t_2, t_3, t_4]$, warunek przyjmuje postać:

$$t_1 + t_3 > t_0 + t_2 + t_4 \quad (1)$$

Przykład: Dla ciągu $A[] = [1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1]$, zgodnie z powyższym kryterium, podciągi spełniające warunek to:

- $T[M] = [1, 130, 1, 9, 11]$, ponieważ $(130 + 9 > 1 + 1 + 11)$
- $T[M] = [1, 9, 11, 6, 1]$, ponieważ $(9 + 6 > 1 + 11 + 1)$
- $T[M] = [1, 1, 3, 1, 1]$, ponieważ $(1 + 1 > 1 + 3 + 1)$

Jak widać, w powyższym przykładzie istnieją trzy podciągi spełniające zadane kryterium.

3 Podstawy teoretyczne.

Algorytmy opiera się na technice przesuwania okna o stałą $M = 5$. Dla każdego możliwego podciągu o długości M w ciągu $A[]$, algorytm sprawdza, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest zapisywany do wyniku.

3.1 Złożoność algorytmu teoretyczna.

Teoretyczna analiza złożoności algorytmu przedstawia się następująco:

- **Złożoność czasowa:** Sprawdzenie jednego okna zajmuje czas stały $\mathcal{O}(1)$. Ponieważ okno przesuwamy przez cały ciąg o długości N , wykonujemy $N - M + 1$ operacji, M jest stałą co powoduje że teoretyczna złożoność powinna wynosić $\mathcal{O}(N)$.
- **Złożoność obliczeniowa:** Algorytm wymaga przechowywania ciągu wejściowego o wielkości N oraz tablicy wyników, która jest alokowana dynamicznie, co teoretyczne w najgorszym przypadku, gdy każdy podciąg spełnia warunek złożoność pamięciowa wynosi $\mathcal{O}(N)$.

4 Szczegóły implementacji.

4.1 Rozwiązanie problemu - podejście pierwsze.

4.1.1 Analiza problemu.

Choć algorytm jest stosunkowo prosty, jego implementacja wymaga uwzględnienia kilku kluczowych aspektów, takich jak efektywne zarządzanie pamięcią, optymalne podejście przy wykorzystaniu pętli oraz optymalizacja operacji sprawdzania warunków, co może znacząco wpłynąć na wydajność końcowego rozwiązania. Pod uwagę trzeba wziąć również obsługę różnych przypadków brzegowych, takich jak bardzo krótkie ciągi wejściowe lub ciągi nie spełniające żadnego z kryteriów co wciąż wpływa na wydajność implementacji.

W podejściu pierwszym, postąpimy w sposób bezpośredni, iterując przez każdy możliwy podciąg o długości M w ciągu $A[N]$ i sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, dodajemy go do listy wyników, liście wyników nie przypisujemy żadnej wartości elementowej ponieważ przyjmujemy, że tablica dynamicznie zmienia swoje rozmiary. Również musimy wziąć pod uwagę przypadki, gdy długość ciągu N jest mniejsza niż M lub dane wejściowe są ułożone w sposób uniemożliwiający znalezienie podciągu spełniającego warunek. W takim przypadku nie ma możliwości znalezienia żadnego podciągu o wymaganej długości, więc algorytm powinien zwrócić pustą listę wyników.

1. Dane wejściowe.

Naszymi danymi wejściowymi są:

- Ciąg liczb całkowitych dodatnich $Tab[]$ również,
- długość ciągu N ,
- stała $M = 5$.

2. Dane wyjściowe.

Naszymi danymi wyjściowymi są:

- Tablica wyników $Wynik[]$, zawierająca wszystkie podciągi.

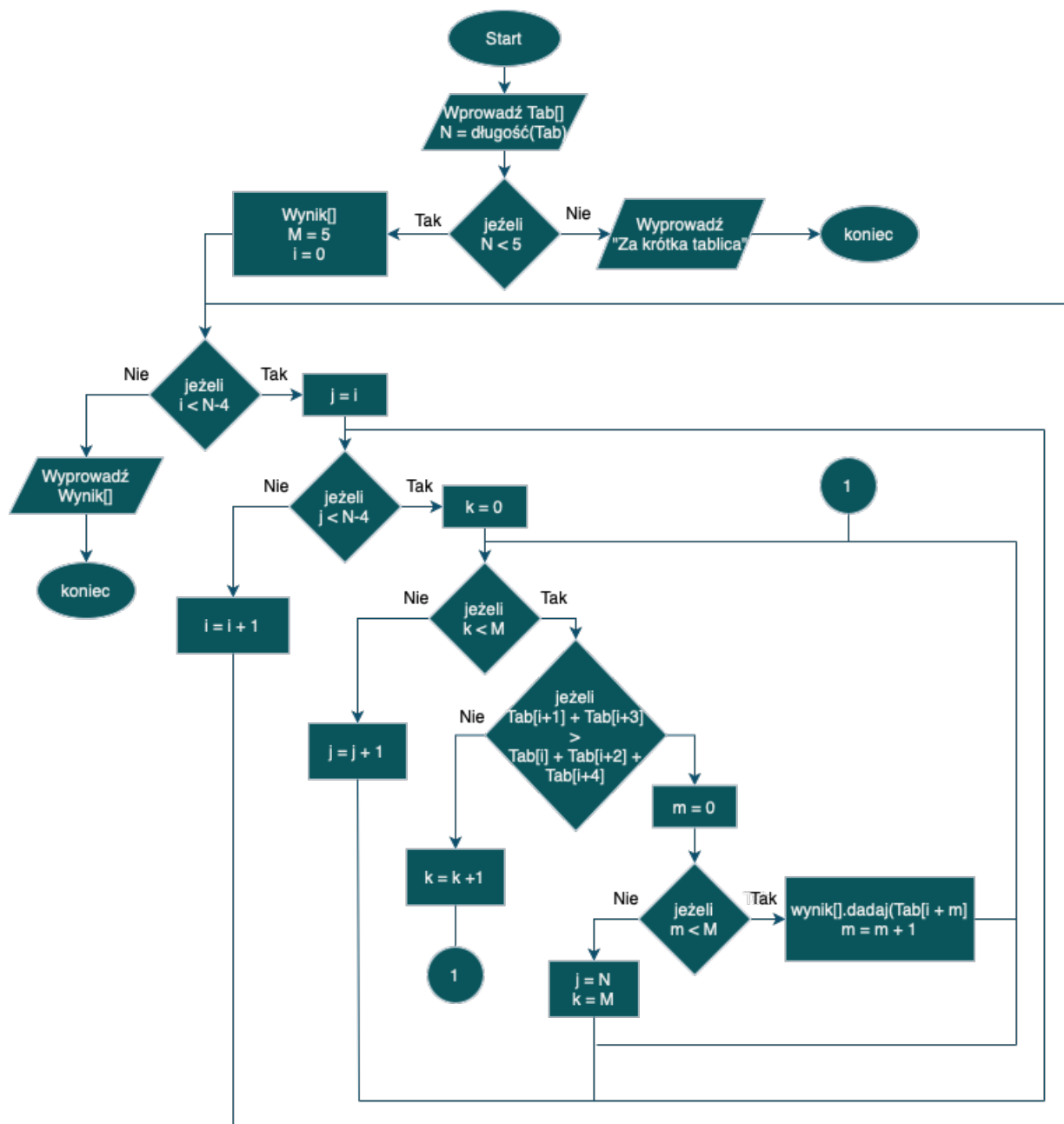
Implementacja wykorzystuje zagnieżdżone pętle iteracyjne (warunki zapętłone), wraz z zmiennymi sterującymi, aby efektywnie przeszukiwać ciąg wejściowy i identyfikować podciągi spełniające określone kryteria. Poniżej przedstawiono szczegółowy opis działania poszczególnych pętli:

- **Pętla zewnętrzna $i < N-4$ (i)** - przesuwa okno przez cały ciąg (od 0 do $N-4$), że względu na warunek sprawdzający (1) musimy pętlę ograniczyć do $N-4$, w celu uniknięcia błędów. Przedstawionej pętli sprawdzamy krok po kroku każdy możliwy podciąg o długości $M = 5$, czy spełnia on warunek z równania (1), również pętla przesuwa okno o jeden element do przodu w każdej iteracji w celu sprawdzenia następnych możliwych podciągów.
- **Pętla wewnętrzna $j < N-4$ (j)** - rozpoczyna się od pozycji $j = i$ i iteruje do $N-4$, sprawdzając warunek z równania (1) dla każdego podciągu zaczynającego się na pozycji i .
- **Pętla iteracyjna $k < M$ (k)** - przechodzi przez elementy podciągu (od 0 do $M-1$), aby obliczyć sumy wymagane do sprawdzenia warunku z równania (1). W tej pętli sumujemy odpowiednie elementy podciągu, aby porównać je zgodnie z kryterium określonym w równaniu (1).
- **Pętla kopiująca $m < M$ (m)** - przepisuje spełniający warunek podciąg do tablicy wyników $Wynik[]$. Ta pętla iteruje przez elementy podciągu i kopiuje je do tablicy wyników, gdy warunek z równania (1) jest spełniony.

Gdy warunek z równania (1) zostanie spełniony dla danego okna, algorytm kopiuje 5 elementów do tablicy wyników i przerywa wewnętrzne pętle poprzez ustawienie wartości sterujących ($j = N$, $k = M$), co wymusza przejście do następnej pozycji okna.

4.1.2 Schemat blokowy algorytmu

Poniżej przedstawiono schemat blokowy ilustrujący działanie algorytmu:



Rys. 1: Schemat blokowy algorytmu wyszukiwania podciągów

4.1.3 Algorytm zapisywany w pseudokodzie.

W poniższym pseudokodzie przedstawiono szczegółową implementację algorytmu. W przypadku pseudokodu, przyjęliśmy wykorzystanie pętli while do iteracji przez elementy ciągu oraz do sprawdzania warunków w porównaniu do schematu blokowego gdzie przyjęliśmy warunki jako pętli.

W pseudokodzie użyto następujących oznaczeń:

- $\text{Tab}[]$ - tablica wejściowa zawierająca ciąg liczb całkowitych dodatnich,
- N - długość tablicy wejściowej,
- $\text{wynik}[]$ - tablica wynikowa przechowująca znalezione podciągi,
- M - stała określająca długość podciągu ($M = 5$),
- i, j, k, m - zmienne sterujące pętli.

```
Input  :  $\text{Tab}[], N$ 
Output:  $\text{wynik}[]$  lub  $-1$ 
if  $N < 5$  then
   $\text{return } -1$ 
 $\text{wynik} \leftarrow []$ ;
const  $M \leftarrow 5$ ;
 $i \leftarrow 0$ ;
while  $i < N - 4$  do
   $j \leftarrow i$ ;
  while  $j < N - 4$  do
     $k \leftarrow 0$ ;
    while  $k < M$  do
      if  $\text{Tab}[i + 1] + \text{Tab}[i + 3] > \text{Tab}[i] + \text{Tab}[i + 2] + \text{Tab}[i + 4]$  then
         $m \leftarrow 0$ ;
        while  $m < M$  do
           $\text{wynik.dodaj}(\text{Tab}[i + m]);$ 
           $m \leftarrow m + 1$ ;
         $j \leftarrow N$ ;
         $k \leftarrow M$ ;
      else
         $k \leftarrow k + 1$ ;
     $j \leftarrow j + 1$ ;
   $i \leftarrow i + 1$ ;
return  $\text{wynik}$ ;
```

Jak widać, algorytm iteruje przez każdy możliwy podciąg o długości M w ciągu $\text{Tab}[]$, sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest dodawany do tablicy wyników $\text{wynik}[]$. W przypadku, gdy długość ciągu N jest mniejsza niż M , algorytm zwraca -1 , wskazując na brak możliwości znalezienia podciągu spełniającego warunek.

4.1.4 Sprawozdanie poprawności algorytmu.

Aby zweryfikować poprawność działania zaimplementowanego algorytmu, przedewszystkim przeprowadzimy test "ołówkowy".

Dla przykładowego ciągu wejściowego $A[] = [5, 20, 5, 30, 5, 2, 1, 2, 1]$

- Długość ciągu $N = 9$,
- Stała $M = 5$.

Krok	i	j	k	m	Sprawdzany warunek	Wynik / Akcja
1	0	0	0	-	$(20 + 30) > (5 + 5 + 5) \rightarrow 50 > 15$	Prawda
2	0	0	0	0-4	Kopiowanie Tab[0...4]	Dodano: [5, 20, 5, 30, 5]
3	0	9	5	-	-	Ustawienie $j = N$, $k = M$ (koniec pętli dla $i = 0$)
4	1	1	0	-	$(5 + 5) > (20 + 30 + 2) \rightarrow 10 > 52$	Fałsz
5	1	1	1	-	-	k zwiększa się o 1
...	1	...	...	-	-	k dochodzi do 5, potem j do $N - 4$
6	2	2	0	-	$(30 + 2) > (5 + 5 + 1) \rightarrow 32 > 11$	Prawda
7	2	2	0	0-4	Kopiowanie Tab[2...6]	Dodano: [5, 30, 5, 2, 1]
8	2	9	5	-	-	Koniec pętli dla $i = 2$
9	3	3	0	-	$(5 + 1) > (30 + 5 + 2) \rightarrow 6 > 37$	Fałsz
10	4	4	0	-	$(2 + 2) > (5 + 1 + 1) \rightarrow 4 > 7$	Fałsz

Tabela 1: Przebieg testu ołówkowego dla ciągu $A[]$

4.1.5 Teoretyczne oszacowanie złożoności obliczeniowej.

Analiza złożoności algorytmu przedstawia się następująco:

W algorytmie pętla i oraz j są zależne od N , natomiast pętla k i m mają stałą liczbę przebiegów ($M = 5$), więc wnoszą stały koszt. Liczbę porównań w gałęzi `if` wyznacza podwójna pętla:

- i przebiega od 0 do $N - 5 \Rightarrow$ około N iteracji,
- dla każdego i zmienna j przechodzi od i do $N - 5 \Rightarrow (N - 4 - i)$ iteracji.

Łączna liczba porównań to więc suma

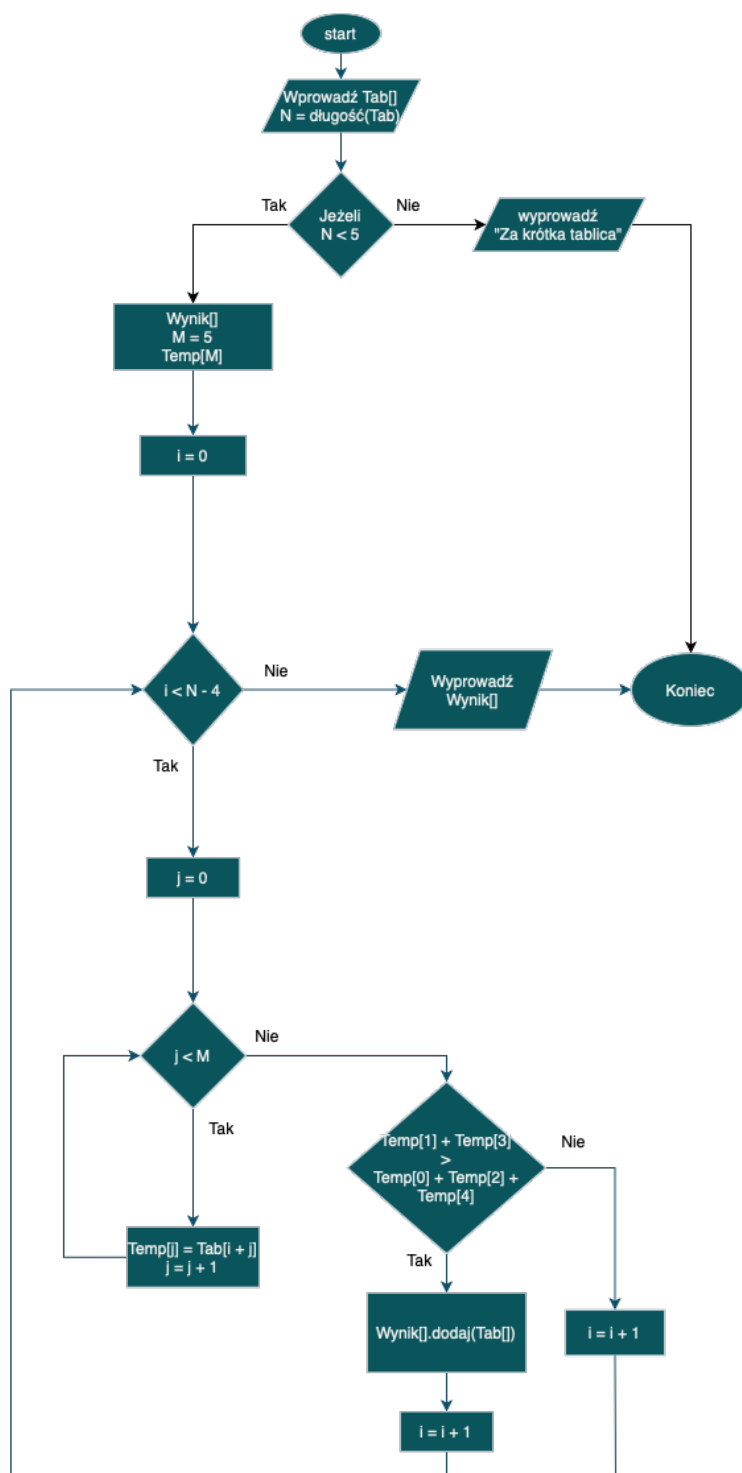
$$\sum_{i=0}^{N-5} (N - 4 - i) = 1 + 2 + \dots + (N - 4) = \frac{(N - 4)(N - 3)}{2} = \mathcal{O}(N^2). \quad (2)$$

Pętla k i m są stałej długości, więc nie zmieniają rzędu złożoności. Całkowita złożoność czasowa algorytmu wynosi zatem $\mathcal{O}(N^2)$.

4.2 Rozwiązanie problemu - podejście drugie.

Po przeanalizowaniu pierwszego podejścia, zauważamy, że jego złożoność czasowa wynosi $\mathcal{O}(N^2)$, co może być nieefektywne dla dużych wartości N . W związku z tym, drugie podejście, które optymalizuje algorytm do złożoności $\mathcal{O}(N)$ poprzez eliminację zbędnych pętli.

4.2.1 Schemat blokowy algorytmu.



Rys. 2: Schemat blokowy algorytmu wyszukiwania podciągów

4.2.2 Algorytm zapisywany w pseudokodzie.

Poniżej przedstawiono szczegółową implementację zoptymalizowanego algorytmu.

W pseudokodzie użyto następujących oznaczeń:

- $\text{Tab}[]$ - tablica wejściowa zawierająca ciąg liczb całkowitych dodatnich,
- N - długość tablicy wejściowej,
- $\text{wynik}[]$ - tablica wynikowa przechowująca znalezione podciągi,
- M - stała określająca długość podciągu ($M = 5$),
- i, j - zmienne sterujące pętlą.

```
Input  :  $\text{Tab}[], N$ 
Output:  $\text{wynik}[]$  lub  $-1$ 
if  $N < 5$  then
  return  $-1$ 
 $\text{wynik} \leftarrow []$ ;
const  $M \leftarrow 5$ ;
 $\text{Temp}[M]$ ;
 $i \leftarrow 0$ ;
while  $i < N - 4$  do
   $j \leftarrow 0$ ;
  while  $j < M$  do
     $\text{Temp}[j] \leftarrow \text{Tab}[i + j]$ ;
     $j \leftarrow j + 1$ ;
  if  $\text{Temp}[1] + \text{Temp}[3] > \text{Temp}[0] + \text{Temp}[2] + \text{Temp}[4]$  then
     $\text{wynik.dodaj}(\text{Temp}[])$ ;
   $i \leftarrow i + 1$ ;
return  $\text{wynik}$ ;
```

4.2.3 Sprawdzenie poprawności algorytmu.

Aby zweryfikować poprawność działania zaimplementowanego algorytmu, również przeprowadzimy test "ołówkowy". Dla przykładowego ciągu wejściowego $A[] = [5, 20, 5, 30, 5, 2, 1, 2, 1]$

- Długość ciągu $N = 9$,
- Stała $M = 5$.

Krok	i	j	$\text{Temp}[]$	Sprawdzany warunek	Wynik / Akcja
1	0	0-4	[5, 20, 5, 30, 5]	$(20 + 30) > (5 + 5 + 5) \rightarrow 50 > 15$	Prawda
2	0	-	-	-	Dodano: [5, 20, 5, 30, 5]
3	1	0-4	[20, 5, 30, 5, 2]	$(5 + 5) > (20 + 30 + 2) \rightarrow 10 > 52$	Fałsz
4	2	0-4	[5, 30, 5, 2, 1]	$(30 + 2) > (5 + 5 + 1) \rightarrow 32 > 11$	Prawda
5	2	-	-	-	Dodano: [5, 30, 5, 2, 1]
6	3	0-4	[30, 5, 2, 1, 2]	$(5 + 1) > (30 + 2 + 1) \rightarrow 6 > 33$	Fałsz
7	4	0-4	[5, 2, 1, 2, 1]	$(2 + 2) > (5 + 1 + 1) \rightarrow 4 > 7$	Fałsz

Tabela 2: Przebieg testu ołówkowego dla ciągu $A[]$

4.2.4 Teoretyczne oszacowanie złożoności obliczeniowej.

Analiza złożoności algorytmu przedstawia się następująco: W algorytmie pętla i jest zależna od N , natomiast pętla j ma stałą liczbę przebiegów ($M = 5$), więc wnoszą stały koszt. Liczbę porównań w gałęzi **if** wyznacza pojedyncza pętla:

- i przebiega od 0 do $N - 5 \Rightarrow$ oznacza to około N iteracji,

Łączna liczba porównań to więc około N . Pętla j jest stałej długości, więc nie zmienia rzędu złożoności. Całkowita złożoność czasowa algorytmu wynosi zatem $\mathcal{O}(N)$.

4.3 Implementacja algorytmów w języku C++ oraz Cmake wraz z Clang++.

4.4 Implementacja

Implementacja obu podejść została przeprowadzona w języku C++. Poniżej przedstawiono kluczowe fragmenty kodu dla obu algorytmów:

4.4.1 Kod źródłowy

```
1  std::vector<int> podciag(int ciag[], int N) {
2      if (N < 5) {
3          return {};
4      }
5      const int M = 5;
6      std::vector<int> wynik;
7      for(int i = 0; i < N - 4; i++) {
8          for(int j = i; j < N - 4; j++) {
9              for(int k = 0; k < M; k++) {
10                 if(ciad[i+1] + ciad[i+3] > ciad[i] + ciad[i+2] + ciad[i+4]) {
11                     for(int m = 0; m < M; m++) {
12                         wynik.push_back(ciad[i + m]);
13                     }
14                     j = N;
15                     k = M;
16                     break;
17                 }
18             }
19         }
20     }
21     return wynik;
22 }
23
24 std::vector<int> podciag_wersja_2(int ciag[], int N) {
25     if (N < 5) {
26         return {};
27     }
28     const int M = 5;
29     int Temp[M];
30     std::vector<int> wynik;
31     for(int i = 0; i < N - 4; i++) {
32         for(int j = 0; j < M; j++) {
33             Temp[j] = ciad[i + j];
34         }
35         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
36             for(int k = 0; k < M; k++) {
37                 wynik.push_back(Temp[k]);
38             }
39         }
40     }
41     return wynik;
42 }
43
44 int main() {
45     int tab[] = {1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1};
46     int N = sizeof(tab)/sizeof(tab[0]);
47     std::vector<int> wynik = podciag(tab, N);
48     wyswietl_wynik(wynik);
49     std::vector<int> wynik2 = podciag_wersja_2(tab, N);
50     wyswietl_wynik(wynik2);
51     return 0;
52 }
```

Jak widać, oba podejścia implementują funkcje `podciag` oraz `podciag_wersja_2`, które realizują opisane algorytmy i zwracają odpowiednie podciągi spełniające warunki.

4.4.2 Testy niewygodnych zestawów danych.

Aby ocenić poprawność zaimplementowanych algorytmów, przeprowadzono testy na różnych zestawach danych wejściowych. Poniżej przedstawiono kod źródłowy testów:

```
1  std::vector<int> podciag(int ciag[], int N) {
2      if (N < 5) {
3          return {};
4      }
5      const int M = 5;
6      std::vector<int> wynik;
7      for(int i = 0; i < N - 4; i++) {
8          for(int j = i; j < N - 4; j++) {
9              for(int k = 0; k < M; k++) {
10                 if(ciad[i+1] + ciad[i+3] > ciad[i] + ciad[i+2] + ciad[i+4]) {
11                     for(int m = 0; m < M; m++) {
12                         wynik.push_back(ciad[i + m]);
13                     }
14                     j = N;
15                     k = M;
16                     break;
17                 }
18             }
19         }
20     }
21     return wynik;
22 }

23
24 std::vector<int> podciag_wersja_2(int ciag[], int N) {
25     if (N < 5) {
26         return {};
27     }
28     const int M = 5;
29     int Temp[M];
30     std::vector<int> wynik;
31     for(int i = 0; i < N - 4; i++) {
32         for(int j = 0; j < M; j++) {
33             Temp[j] = ciad[i + j];
34         }
35         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
36             for(int k = 0; k < M; k++) {
37                 wynik.push_back(Temp[k]);
38             }
39         }
40     }
41     return wynik;
42 }

43
44 void wyswietl_dane(int tablica[], int N) {
45     std::cout << "Ciag wejsciowy: ";
46     for (int i = 0; i < N; i++) {
47         std::cout << tablica[i] << " ";
48     }
49     std::cout << std::endl;
50 }

51
52 void wyswietl_wynik(std::vector<int> wynik) {
53     if(!wynik.empty()) {
54         int k{0};
55         for (int i = 0; i < wynik.size() / 5; i++) {
56             std::cout << "[ ";
57             for (int j = 0; j < 5; j++) {
58                 std::cout << wynik[j+k] << " ";
59             }
60         }
61     }
```

```

60         std::cout << "]";
61         if (i < wynik.size() / 5 - 1) std::cout << ", ";
62         k += 5;
63     }
64     std::cout << std::endl;
65 }
66 else {
67     std::cout << "Ciag jest za krotki lub nie ma zadnego spelnionego ciagu\n";
68 }
69 }
70
71 struct Wynik {
72     std::vector<int> p1;
73     std::vector<int> p2;
74 };
75
76 void test_niewygodnych_zestawow() {
77     int tab1[] = {1, 3, 2, 3};
78     int tab2[] = {7, 7, 7, 7, 7};
79     int tab3[] = {1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1};
80     int tab4[] = {1, 100, 1, 100, 1, 2, 2, 2, 2};
81     int tab5[] = {5, 200, 5, 200, 5, 10, 10, 10, 10};
82
83     struct Test {
84         int* tablica;
85         int rozmiar;
86     };
87
88     Test testy[] = {
89         {tab1, sizeof(tab1)/sizeof(tab1[0])},
90         {tab2, sizeof(tab2)/sizeof(tab2[0])},
91         {tab3, sizeof(tab3)/sizeof(tab3[0])},
92         {tab4, sizeof(tab4)/sizeof(tab4[0])},
93         {tab5, sizeof(tab5)/sizeof(tab5[0])}
94     };
95
96     for (int i = 0; i < 5; i++) {
97         std::cout << "\n== TEST " << (i+1) << " == " << std::endl;
98         std::cout << "Wejscie: ";
99         std::cout << std::endl;
100         wyswietl_dane(testy[i].tablica, testy[i].rozmiar);
101
102         Wynik wynik = test_zestawow(
103             std::vector<int>(testy[i].tablica, testy[i].tablica + testy[i].rozmiar
104             ),
105             testy[i].rozmiar
106         );
107
108         std::cout << std::endl;
109         std::cout << "Algorytm 1: ";
110         wyswietl_wynik(wynik.p1);
111         std::cout << "Algorytm 2: ";
112         wyswietl_wynik(wynik.p2);
113     }
114
115 int main() {
116     test_niewygodnych_zestawow();
117     return 0;
118 }

```

Wyniki testów:

==== TEST 1 ==== Wejście: Ciąg wejściowy: 1 3 2 3

Algorytm 1: Ciąg jest za krótki lub nie ma żadnego spełnionego ciągu

Algorytm 2: Ciąg jest za krótki lub nie ma żadnego spełnionego ciągu

==== TEST 2 ==== Wejście: Ciąg wejściowy: 7 7 7 7 7

Algorytm 1: Ciąg jest za krótki lub nie ma żadnego spełnionego ciągu

Algorytm 2: Ciąg jest za krótki lub nie ma żadnego spełnionego ciągu

==== TEST 3 ==== Wejście: Ciąg wejściowy: 1 130 1 9 11 6 1 1 1 3 1

Algorytm 1: [1 130 1 9 11], [1 9 11 6 1], [1 1 1 3 1]

Algorytm 2: [1 130 1 9 11], [1 9 11 6 1], [1 1 1 3 1]

==== TEST 4 ==== Wejście: Ciąg wejściowy: 1 100 1 100 1 2 2 2 2

Algorytm 1: [1 100 1 100 1], [1 100 1 2 2]

Algorytm 2: [1 100 1 100 1], [1 100 1 2 2]

==== TEST 5 ==== Wejście: Ciąg wejściowy: 5 200 5 200 5 10 10 10 10

Algorytm 1: [5 200 5 200 5], [5 200 5 10 10]

Algorytm 2: [5 200 5 200 5], [5 200 5 10 10]

4.4.3 Test wydajnościowy algorytmów.

```
1
2 std::vector<int> podciag(int ciag[], int N) {
3     if (N < 5) {
4         return {};
5     }
6     const int M = 5;
7     std::vector<int> wynik;
8     for(int i = 0; i < N - 4; i++) {
9         for(int j = i; j < N - 4; j++) {
10            for(int k = 0; k < M; k++) {
11                if(ciag[i+1] + ciag[i+3] > ciag[i] + ciag[i+2] + ciag[i+4]) {
12                    for(int m = 0; m < M; m++) {
13                        wynik.push_back(ciag[i + m]);
14                    }
15                    j = N;
16                    k = M;
17                    break;
18                }
19            }
20        }
21    }
22    return wynik;
23 }
24
25 std::vector<int> podciag_wersja_2(int ciag[], int N) {
26     if (N < 5) {
27         return {};
28     }
29     const int M = 5;
30     int Temp[M];
31     std::vector<int> wynik;
32     for(int i = 0; i < N - 4; i++) {
33         for(int j = 0; j < M; j++) {
34             Temp[j] = ciag[i + j];
35         }
36         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
37             for(int k = 0; k < M; k++) {
38                 wynik.push_back(Temp[k]);
39             }
40         }
41     }
42     return wynik;
43 }
```

```

44
45 void generuj_dane(int ciag[], int N) {
46     std::srand(std::time(NULL));
47     for (int i = 0; i < N; i++) {
48         ciag[i] = rand() % 150 + 1; // rand() % (max - min + 1) + min
49     }
50 }
51
52 void wyswietl_dane(int tablica[], int N) {
53     std::cout << "Ciag wejsciowy: ";
54     for (int i = 0; i < N; i++) {
55         std::cout << tablica[i] << " ";
56     }
57     std::cout << std::endl;
58 }
59
60 void wyswietl_wynik(std::vector<int> wynik) {
61     if(!wynik.empty()) {
62         int k{0};
63         for (int i = 0; i < wynik.size() / 5; i++) {
64             std::cout << "[ ";
65             for (int j = 0; j < 5; j++) {
66                 std::cout << wynik[j+k] << " ";
67             }
68             std::cout << "];";
69             if (i < wynik.size() / 5 - 1) std::cout << ", ";
70             k += 5;
71         }
72         std::cout << std::endl;
73     }
74     else {
75         std::cout << "Ciag jest za krotki lub nie ma zadnego spelnionego ciagu\n";
76     }
77 }
78
79 void zapis_do_pliku(std::vector<int> wynik, const std::string& filename) {
80     std::ofstream file(filename);
81     if (file.is_open()) {
82         if(!wynik.empty()) {
83             int k{0};
84             for (int i = 0; i < wynik.size() / 5; i++) {
85                 file << "[ ";
86                 for (int j = 0; j < 5; j++) {
87                     file << wynik[j+k] << " ";
88                 }
89                 file << "];";
90                 if (i < wynik.size() / 5 - 1) file << ", ";
91                 k += 5;
92             }
93         }
94     }
95 }
96
97 struct Wynik {
98     std::vector<int> p1;
99     std::vector<int> p2;
100 };
101
102 Wynik test_zestawow(std::vector<int> tab, int N) {
103     std::vector<int> wynik = podciag(tab.data(), N);
104     std::vector<int> wynik2 = podciag_wersja_2(tab.data(), N);
105     return {wynik, wynik2};
106 }

```

```

107
108 void test_wydajnosci() {
109     //Utworzenie zmiennych startowych
110     int N[] = {2500, 5000, 10000, 20000, 30000, 40000, 50000, 60000, 70000,
111     80000};
112     int liczba_testow = sizeof(N)/sizeof(N[0]);
113     double* czas1 = new double[liczba_testow];
114     double* czas2 = new double[liczba_testow];
115
116     for(int i = 0; i < liczba_testow; i++) {
117         std::vector<int> ciag_wejscowy(N[i]);
118         std::vector<int> wynik(N[i]);
119
120         //Generowanie danych
121         generuj_dane(ciag_wejscowy.data(), N[i]);
122         // wyswietl_dane(ciag_wejscowy.data(), N[i]);
123
124         clock_t start1 = clock();
125         std::vector<int> wynik1 = podciag(ciag_wejscowy.data(), N[i]);
126         clock_t stop1 = clock();
127         czas1[i] = (double)(stop1 - start1) / CLOCKS_PER_SEC;
128
129         clock_t start2 = clock();
130         std::vector<int> wynik2 = podciag_wersja_2(ciag_wejscowy.data(), N[i]);
131         clock_t stop2 = clock();
132         czas2[i] = (double)(stop2 - start2) / CLOCKS_PER_SEC;
133     }
134     std::cout << "L.p.   n   t1[s]   t2[s]" << "\n";
135     for(int i = 0; i < liczba_testow; i++) {
136         std::cout << std::fixed << std::setprecision(6) << i+1 << "   " << N[i] <<
137         "   " << czas1[i] << "   " << czas2[i] << '\n';
138     }
139
140     std::string filename = "wyniki_wydajnosci.txt";
141     std::ofstream file;
142     file.open("wyniki_wydajnosci.txt");
143     file << "L.p.   n   t1[s]   t2[s]" << "\n";
144     for(int i = 0; i < liczba_testow; i++) {
145         file << std::fixed << std::setprecision(6) << i+1 << "   " << N[i] << "
146         " << czas1[i] << "   " << czas2[i] << '\n';
147     }
148     file.close();
149
150     delete[] czas1;
151     delete[] czas2;
152 }
153
154 int main () {
155     test_wydajnosci();
156
157     return 0;
158 }

```


Wyniki testów wydajnościowych przedstawiono w poniższej tabeli:

L.p.	n	t_1 [s]	t_2 [s]
1	2500	0.019769	0.000061
2	5000	0.076027	0.000111
3	10000	0.299872	0.000221
4	20000	1.176934	0.000419
5	30000	2.614826	0.000590
6	40000	4.722152	0.000810
7	50000	7.379533	0.001028
8	60000	10.462567	0.001233
9	70000	14.272821	0.001409
10	80000	18.639339	0.001573

Tabela 3: Wyniki testów wydajnościowych